Università del Salento
Facoltà di Ingegneria
Corso di Laurea in Ingegneria dell'Informazione

ChillChain — Piattaforma IoT per il trasporto refrigerato condiviso

Matching domanda-offerta e rilevazione anomalie con LSTM Autoencoder

Relatore
Prof. Luigi Patrono

Tutor
Ing. Teodoro Montanaro

Studente
Antonio Bello
Mat. 20093232

Anno Accademico 2025–2026

Indice

1 Introduzione **4**

2 Analisi dei Requisiti **6**
2.1 Requisiti Funzionali 6
2.1.1 Gestione degli utenti e autenticazione 6
2.1.2 Gestione della flotta 6
2.1.3 Gestione dei viaggi e delle spedizioni 7
2.1.4 Ricerca e prenotazione delle spedizioni 7
2.1.5 Monitoraggio telemetrico in tempo reale 7
2.1.6 Rilevazione delle anomalie 7
2.1.7 Gestione delle notifiche 8
2.2 Requisiti Non Funzionali 8
2.2.1 Usabilità 8
2.2.2 Sicurezza 8
2.2.3 Modularità e manutenibilità 8
2.2.4 Portabilità e riproducibilità del deployment 8

3 Stato dell'Arte **9**
3.1 Flock Freight – FlockDirect® Shared Truckload 9
3.2 Quicargo – Piattaforma collaborativa con pricing dinamico . . 10
3.3 Gillespie et al. – Rilevazione anomalie IoT nel trasporto refrigerato 10
3.4 Sgueglia et al. – Systematic literature review su anomaly detection IoT 11
3.5 Sintesi e posizionamento della soluzione proposta 11

4 Tecnologie Utilizzate **13**
4.1 Spring Boot 13
4.2 React e TypeScript 13
4.3 MQTT e Mosquitto 14
4.4 MongoDB 14

4.5 Python, FastAPI e PyTorch 15
4.6 Google Maps Platform 15
4.7 Docker e Docker Compose 16

5 Soluzione Proposta 17
5.1 Architettura del sistema 17
5.2 Gestione degli utenti e autenticazione 18
5.3 Gestione della flotta e dei viaggi 19
5.3.1 Aggiunta di un veicolo 19
5.3.2 Lista veicoli 19
5.3.3 Lista viaggi 20
5.4 Marketplace: ricerca dei viaggi disponibili e creazione della spedizione 20
5.4.1 Fase 1 — Ricerca dei veicoli disponibili 20
5.4.2 Fase 2 — Ricerca dei viaggi già programmati 21
5.4.3 Fase 3 — Validazione geografica dei viaggi già programmati 21
5.4.4 Fase 4 — Ricalcolo del percorso con waypoint 21
5.4.5 Fase 5 — Pricing con divisore esponenziale 21
5.4.6 Selezione e conferma della tratta 22
5.5 Generazione dei dati telemetrici 22
5.6 Pipeline di anomaly detection 22
5.6.1 Addestramento del modello 23
5.6.2 Inferenza in tempo reale 23
5.6.3 Smoothing con contatori consecutivi 23
5.7 Gestione delle notifiche 24
5.8 Dashboard telemetrica 24

6 Validazione Funzionale 25
6.1 Ambiente di test 25
6.2 Scenario di validazione 25
6.2.1 Fase 1 — Configurazione della flotta (Amministratore) 26
6.2.2 Fase 2 — Prima spedizione: creazione della tratta (Cliente A) 26
6.2.3 Fase 3 — Seconda spedizione: condivisione della tratta (Cliente B) 27
6.2.4 Fase 4 — Visualizzazione dei viaggi programmati e avvio simulazione 28
6.2.5 Fase 5 — Rilevazione anomalia e notifica (Tecnico) . . 28

7 Conclusioni e Sviluppi Futuri **33**
7.1 Conclusioni 33
7.2 Sviluppi futuri 33
7.2.1 Ottimizzazione temporale e scheduling attivo 33
7.2.2 Scalabilità su piattaforme cloud 34
7.2.3 Miglioramento della sensibilità dell'anomaly detection . 34
7.2.4 Indicizzazione spaziale con PostGIS 34
7.2.5 Portale cliente per il tracking delle spedizioni 34
7.2.6 Sistema di notifiche push per il tecnico 35

Capitolo 1

Introduzione

La gestione della catena del freddo rappresenta una delle sfide più critiche nel settore della logistica moderna. Il trasporto di merci deperibili richiede il mantenimento rigoroso di condizioni ambientali controllate lungo l'intera filiera distributiva, dalla produzione alla consegna finale. Anche una breve interruzione della temperatura può compromettere irrimediabilmente la qualità e la sicurezza dei prodotti trasportati, con conseguenze economiche rilevanti e potenziali rischi per la salute pubblica. A rendere il problema ancora più complesso contribuisce la frammentazione del mercato delle spedizioni refrigerate: da un lato, le aziende di trasporto gestiscono flotte di veicoli che spesso viaggiano con capacità parzialmente inutilizzata; dall'altro, i clienti che necessitano di spedire piccoli quantitativi si trovano costretti a sostenere costi elevati per noleggiare un intero veicolo refrigerato o a ricorrere a soluzioni LTL (*Less Than Truckload*) tradizionali, che comportano molteplici passaggi in magazzino, tempi di consegna più lunghi e un rischio maggiore di rottura della catena del freddo.

Le soluzioni attualmente disponibili sul mercato affrontano questi problemi in modo parziale e disgiunto. Piattaforme come Flock Freight hanno introdotto il concetto di *Shared Truckload* (STL), sfruttando algoritmi di intelligenza artificiale per consolidare spedizioni di clienti diversi su uno stesso veicolo e ottimizzare le rotte in maniera automatica. Queste piattaforme adottano un approccio flessibile in cui l'algoritmo può combinare spedizioni con percorsi anche molto diversi, e sono progettate per il trasporto merci generico senza integrare funzionalità di monitoraggio della catena del freddo né strumenti di rilevazione anomalie per spedizioni refrigerate. Sul versante del monitoraggio, il panorama delle soluzioni IoT per la *cold chain* è maturo e in forte espansione. Ciò nonostante, queste soluzioni operano tipicamente come sistemi B2B verticali, scollegati dalla fase commerciale di prenotazione e pricing, e si limitano per lo più a un monitoraggio basato su soglie statiche

che, pur essendo semplice da implementare, non è in grado di rilevare pattern anomali complessi o degradi graduali delle prestazioni dell'impianto di refrigerazione.

Il presente lavoro propone **ChillChain**, una piattaforma IoT che integra in un'unica soluzione il matching tra domanda e offerta di spedizioni refrigerate e il monitoraggio intelligente della catena del freddo con rilevazione automatica delle anomalie. L'idea alla base del progetto è semplice: consentire ai clienti di visualizzare le tratte già programmate dai trasportatori, valutarne la compatibilità con le proprie esigenze di spedizione e aggregarsi a un viaggio esistente. La rotta del veicolo viene ricalcolata dinamicamente per includere le nuove fermate, mentre un sistema di pricing basato sull'occupazione del mezzo applica un divisore esponenziale che rende il costo per singolo cliente progressivamente più conveniente all'aumentare delle spedizioni condivise, incentivando così un utilizzo più efficiente della capacità di trasporto.

Accanto alla componente di marketplace, la piattaforma integra un sistema di monitoraggio in tempo reale dei parametri operativi dei veicoli refrigerati. I dati telemetrici vengono trasmessi via MQTT e analizzati da un servizio dedicato di rilevazione anomalie. Quest'ultimo impiega un LSTM Autoencoder, addestrato sul comportamento nominale dell'impianto, per calcolare l'errore di ricostruzione di ciascuna finestra temporale di osservazione: quando tale errore supera una soglia appresa in fase di training, il sistema genera una notifica che viene inoltrata al tecnico responsabile attraverso la dashboard della piattaforma.

Dal punto di vista architetturale, ChillChain adotta un'architettura a microservizi composta da un backend Spring Boot che gestisce la logica applicativa e le API REST, un frontend React con TypeScript per l'interfaccia utente, un broker MQTT Mosquitto per la comunicazione in tempo reale, due servizi Python dedicati rispettivamente allo streaming dei dati telemetrici e alla rilevazione delle anomalie, e un database MongoDB per la persistenza. L'intera infrastruttura è orchestrata tramite Docker Compose, garantendo riproducibilità e semplicità di deployment. L'obiettivo del progetto è dimostrare la fattibilità di un approccio integrato che unisca la dimensione commerciale della logistica collaborativa con quella tecnica del monitoraggio predittivo, ponendo particolare attenzione all'usabilità dell'interfaccia e alla chiarezza con cui le informazioni vengono presentate ai diversi attori del sistema: clienti, amministratori e tecnici.

Capitolo 2

Analisi dei Requisiti

In questo capitolo vengono descritti i requisiti funzionali e non funzionali che il sistema ChillChain è chiamato a soddisfare. L'analisi è stata condotta considerando i tre profili utente previsti dalla piattaforma — amministratore, cliente e tecnico — e le esigenze operative che ciascuno di essi esprime nei confronti del sistema.

2.1 Requisiti Funzionali

I requisiti funzionali definiscono le capacità operative che la piattaforma deve offrire ai propri utenti e descrivono il comportamento atteso del sistema in risposta a specifiche azioni o eventi.

2.1.1 Gestione degli utenti e autenticazione

Il sistema deve consentire l'autenticazione mediante credenziali protette. A ogni utente viene assegnato un ruolo tra amministratore, cliente e tecnico. L'accesso alle funzionalità della piattaforma deve essere regolato in base a tale ruolo.

2.1.2 Gestione della flotta

L'amministratore deve poter registrare nuovi veicoli nella piattaforma, specificando per ciascuno le dimensioni del vano di carico (larghezza, altezza, lunghezza), la portata massima in peso, il costo per chilometro e l'indicazione sulla presenza o meno dell'impianto di refrigerazione. Il sistema deve consentire la visualizzazione dell'intera flotta e la rimozione dei veicoli non più operativi.

2.1.3 Gestione dei viaggi e delle spedizioni

L'amministratore deve poter visualizzare l'elenco dei viaggi programmati e consultare le spedizioni associate a ciascun viaggio.

2.1.4 Ricerca e prenotazione delle spedizioni

Il cliente deve poter ricercare i viaggi disponibili inserendo l'indirizzo di partenza, l'indirizzo di arrivo, le caratteristiche del proprio pacco — dimensioni, peso, necessità di refrigerazione — e la data entro cui la consegna deve essere completata. Il sistema deve restituire due categorie di risultati: nuovi viaggi, generati a partire dai veicoli della flotta ancora privi di viaggi assegnati con percorso calcolato ad hoc, e viaggi condivisi, corrispondenti a viaggi già programmati che dispongano di capacità residua sufficiente e di compatibilità geografica con il percorso richiesto. Per i viaggi condivisi, il prezzo deve essere ricalcolato applicando un meccanismo di divisione basato sul numero di spedizioni già presenti, in modo da incentivare la condivisione del veicolo. Il cliente seleziona il viaggio desiderato tra quelli proposti. Alla conferma della selezione, il sistema deve: nel caso di un nuovo viaggio, crearlo automaticamente con il percorso calcolato; nel caso di un viaggio condiviso, ricalcolare dinamicamente il percorso introducendo le fermate del nuovo cliente come tappe intermedie.

2.1.5 Monitoraggio telemetrico in tempo reale

Il sistema deve acquisire i dati telemetrici provenienti dai sensori installati a bordo dei veicoli refrigerati. Il tecnico deve poter visualizzare i dati telemetrici attraverso una dashboard che presenti l'andamento temporale dei parametri mediante grafici, con la possibilità di filtrare per veicolo e per tipologia di dato telemetrico.

2.1.6 Rilevazione delle anomalie

Un servizio dedicato deve analizzare in maniera continuativa i flussi telemetrici provenienti dai veicoli e identificare automaticamente condizioni anomale. Quando un'anomalia viene confermata, il sistema deve generare una notifica persistente, associata al veicolo interessato.

2.1.7 Gestione delle notifiche

Il tecnico deve poter consultare l'elenco delle notifiche ricevute, ciascuna contenente il nome del veicolo, il timestamp dell'evento, l'errore di ricostruzione e un messaggio descrittivo dell'anomalia rilevata.

2.2 Requisiti Non Funzionali

I requisiti non funzionali specificano le qualità architetturali e operative che il sistema deve garantire trasversalmente rispetto alle singole funzionalità.

2.2.1 Usabilità

L'interfaccia utente deve essere progettata con particolare attenzione alla chiarezza e all'immediatezza d'uso. Le informazioni devono essere presentate in modo differenziato in base al ruolo dell'utente.

2.2.2 Sicurezza

Il sistema deve proteggere le comunicazioni tra client e server mediante autenticazione e assegnazione dei ruoli agli utenti autenticati. Le password degli utenti devono essere memorizzate in forma cifrata.

2.2.3 Modularità e manutenibilità

L'architettura del sistema deve seguire il paradigma a microservizi, con una separazione netta tra le componenti applicative. Ogni componente deve poter essere sviluppato, testato e aggiornato indipendentemente dagli altri.

2.2.4 Portabilità e riproducibilità del deployment

L'intera infrastruttura deve essere containerizzata tramite Docker e orchestrata mediante Docker Compose, in modo da garantire che il sistema possa essere avviato in maniera identica su qualsiasi macchina che disponga del runtime Docker.

Capitolo 3

Stato dell'Arte

In questo capitolo vengono analizzate le principali soluzioni esistenti nel panorama della logistica collaborativa, del monitoraggio IoT della catena del freddo e della rilevazione di anomalie su serie temporali. Per ciascuna soluzione viene fornita una descrizione sintetica, seguita da un'analisi critica che ne evidenzia i limiti rispetto alla proposta presentata in questo lavoro.

3.1 Flock Freight – FlockDirect® Shared Truckload

Flock Freight è un'azienda statunitense che ha introdotto il concetto di *Shared Truckload* (STL), una modalità di spedizione che consente di consolidare carichi di clienti diversi su un unico camion mediante una tecnologia brevettata basata su intelligenza artificiale. L'algoritmo di Flock Freight analizza posizioni geografiche, calendari di spedizione e dimensioni dei carichi per individuare, tra miliardi di combinazioni possibili, gli abbinamenti più efficienti, instradando il veicolo su percorsi diretti e privi di soste in magazzini intermedi [2]. A differenza del tradizionale LTL (*Less Than Truckload*), in cui la merce viene scaricata e ricaricata più volte lungo il percorso, nello STL il carico non lascia mai il veicolo fino alla consegna, riducendo drasticamente il rischio di danneggiamento e i tempi di transito. Flock Freight dichiara consegne puntuali nel 97% dei casi e un tasso di danneggiamento 5,4 volte inferiore rispetto all'LTL, con una riduzione delle emissioni di CO_2 fino al 40%.

Nonostante la solidità della soluzione, il modello di Flock Freight presenta caratteristiche differenti rispetto all'approccio proposto in ChillChain. Dal punto di vista dell'ottimizzazione, Flock adotta un approccio flessibile basato su AI che può combinare spedizioni con percorsi anche molto diversi

tra loro, massimizzando il numero di abbinamenti possibili sull'intera rete. ChillChain segue invece un approccio spazialmente più rigido: le spedizioni condivise devono trovarsi lungo il percorso già programmato del veicolo, con una tolleranza massima di deviazione sia in termini di distanza dalla rotta sia di incremento della durata del viaggio. Questo vincolo riduce il numero di combinazioni possibili, ma quando il matching avviene il risparmio è potenzialmente più elevato, poiché la deviazione introdotta è minima e la riduzione di prezzo scala in modo aggressivo con il numero di spedizioni a bordo. Si tratta di due approcci alternativi, ciascuno con i propri compromessi tra flessibilità e ottimizzazione del costo. Inoltre, la piattaforma Flock Freight è progettata per il trasporto merci generico e non integra alcuna funzionalità di monitoraggio della catena del freddo: non offre strumenti di telemetria né di rilevazione anomalie per spedizioni refrigerate.

3.2 Quicargo – Piattaforma collaborativa con pricing dinamico

Quicargo è una piattaforma logistica digitale fondata nei Paesi Bassi che opera come intermediario tra mittenti e trasportatori nel segmento LTL. La piattaforma consente ai clienti di prenotare spedizioni di pallet verso 29 Paesi europei [4]. Un aspetto particolarmente rilevante dal punto di vista scientifico è il lavoro di Atasoy, Schulte e Steenkamp (2020), che hanno studiato il caso Quicargo formulando un problema collaborativo di *pickup and delivery* con finestre temporali (PDPTW), in cui il pricing pagato dalla piattaforma ai trasportatori viene modulato dinamicamente per bilanciare gli interessi di tutti gli attori coinvolti [1].

Quicargo opera esclusivamente nel trasporto merci generico. Il trasporto di prodotti refrigerati non è disponibile. ChillChain offre invece un servizio dedicato al trasporto refrigerato.

3.3 Gillespie et al. – Rilevazione anomalie IoT nel trasporto refrigerato

Gillespie et al. (2023) propongono un sistema IoT per la rilevazione in tempo reale di anomalie durante il trasporto refrigerato di prodotti alimentari deperibili, pubblicato sulla rivista *Sustainability* [3]. Il sistema si basa sul data logger cellulare Digital Matter Eagle, collegato a due sonde di temperatura posizionate all'interno del vano refrigerato del veicolo. I dati vengono

trasmessi a una piattaforma cloud, dove un algoritmo di alerting basato su soglie di temperatura genera notifiche destinate al personale di magazzino e ai conducenti.

Il lavoro di Gillespie et al. rappresenta un riferimento importante per il monitoraggio IoT applicato alla catena del freddo, in quanto dimostra la fattibilità di un sistema di alerting in tempo reale a basso costo basato su hardware commerciale. Tuttavia, la soluzione presenta un limite dal punto di vista della rilevazione delle anomalie: il sistema si basa esclusivamente su soglie statiche di temperatura. Questo approccio, pur essendo semplice e robusto, non è in grado di identificare pattern anomali complessi. ChillChain propone l'utilizzo di un modello LSTM Autoencoder per la rilevazione di anomalie multivariate per la cattura di anomalie complesse.

3.4 Sgueglia et al. – Systematic literature review su anomaly detection IoT

Sgueglia, Di Sorbo, Visaggio e Canfora (2022) presentano una revisione sistematica della letteratura sulle soluzioni di rilevazione anomalie per serie temporali in ambienti IoT, pubblicata sulla rivista *Future Generation Computer Systems* [6]. Lo studio analizza 62 articoli pubblicati tra il 2014 e il 2021, mappando i metodi e le tecniche adottati dalla comunità scientifica per affrontare le problematiche specifiche della rilevazione di anomalie su dati generati da dispositivi IoT. La survey esamina in particolare tre aspetti critici: la riduzione della dimensionalità dei dati sensoristici, la localizzazione delle anomalie all'interno delle serie temporali e le sfide legate al monitoraggio in tempo reale sotto vincoli di risorse computazionali e temporali. Gli autori classificano gli approcci in base alla tipologia di algoritmo impiegato — metodi statistici, tecniche di machine learning tradizionale e architetture di deep learning — evidenziando come gli approcci basati su autoencoder e reti ricorrenti (LSTM) abbiano mostrato risultati particolarmente promettenti per la rilevazione di anomalie su serie temporali multivariate.

3.5 Sintesi e posizionamento della soluzione proposta

Dall'analisi dello stato dell'arte emerge che le soluzioni esistenti affrontano in modo separato i tre ambiti coinvolti nel presente lavoro: il matching tra domanda e offerta di spedizioni, il monitoraggio IoT della catena del fred-

do e la rilevazione di anomalie basata su deep learning. Le piattaforme di trasporto collaborativo come Flock Freight e Quicargo offrono meccanismi di consolidamento dei carichi ma non integrano funzionalità di monitoraggio per spedizioni refrigerate. Il sistema IoT di Gillespie et al. dimostra l'efficacia del monitoraggio in tempo reale ma si limita a due sonde di temperatura con soglie statiche, senza sfruttare le potenzialità del machine learning per la rilevazione di anomalie multivariate.

La review sistematica di Sgueglia et al. conferma come gli approcci basati su LSTM Autoencoder rappresentino attualmente una delle tecniche più promettenti per l'anomaly detection in serie temporali IoT multivariate, evidenziando tuttavia che la maggior parte dei lavori considerati propone componenti analitici isolati, senza integrazione in piattaforme operative complete [6].

ChillChain si propone di colmare questo gap attraverso una piattaforma integrata che combina un marketplace con pricing dinamico basato sull'occupazione del veicolo, un sistema di monitoraggio in tempo reale e un servizio di rilevazione anomalie basato su LSTM Autoencoder.

Capitolo 4

Tecnologie Utilizzate

In questo capitolo vengono descritte le principali tecnologie impiegate nella realizzazione di ChillChain, evidenziando per ciascuna gli aspetti specifici che hanno contribuito alla soluzione adottata.

4.1 Spring Boot

Spring Boot è il framework Java utilizzato per la realizzazione del backend della piattaforma. La scelta è ricaduta su questo framework per la sua capacità di semplificare la configurazione di applicazioni.

Nel contesto di ChillChain, Spring Boot è stato impiegato per esporre le API REST che gestiscono l'intero ciclo di vita delle entità di dominio: veicoli, viaggi, spedizioni, notifiche e utenti. Il modulo Spring Security è stato configurato per implementare l'autenticazione basata su token JWT. L'integrazione con MongoDB è stata realizzata tramite Spring Data MongoDB, che consente di definire repository dichiarativi con query derivate dal nome dei metodi e pipeline di aggregazione personalizzate. Un aspetto particolarmente rilevante è l'integrazione con il protocollo MQTT tramite Spring Integration: il backend sottoscrive i topic di telemetria e anomalie pubblicati dai servizi Python, riceve i messaggi attraverso channel adapter dedicati e li instrada a service activator che si occupano della persistenza dei dati e della generazione delle notifiche.

4.2 React e TypeScript

Il frontend della piattaforma è stato realizzato con React, una libreria JavaScript per la costruzione di interfacce utente basate su componenti, in com-

binazione con TypeScript, un superset tipizzato di JavaScript che aggiunge il controllo statico dei tipi.

L'impiego di React ha permesso di strutturare l'interfaccia come un insieme di componenti riutilizzabili, ciascuno responsabile di una porzione specifica della vista. La gestione dello stato applicativo si basa sugli hook nativi di React (`useState`, `useEffect`) e su `localStorage` per la persistenza delle informazioni di sessione attraverso la navigazione. La visualizzazione dei dati telemetrici in tempo reale è stata realizzata con la libreria Recharts. La visualizzazione cartografica delle tratte si appoggia a React-Leaflet, impiegato per il rendering delle polyline dei percorsi su mappa interattiva. L'intero progetto frontend utilizza Vite come build tool e Tailwind CSS come framework di utility per lo stile, che ha permesso di ottenere un'interfaccia coerente e responsive.

4.3 MQTT e Mosquitto

MQTT (*Message Queuing Telemetry Transport*) è il protocollo di comunicazione adottato per lo scambio di dati telemetrici tra i servizi della piattaforma. Si tratta di un protocollo di messaggistica leggero basato sul pattern *publish/subscribe*.

In ChillChain, MQTT costituisce il tessuto connettivo tra tre attori: il servizio di streaming (publisher), il servizio di anomaly detection (subscriber e publisher) e il backend Spring Boot (subscriber). Il broker Mosquitto gestisce l'instradamento dei messaggi attraverso una struttura gerarchica di topic. Questa architettura disaccoppia completamente i produttori dai consumatori dei dati. La riconnessione automatica, configurata sia lato Python (tramite la libreria Paho MQTT) sia lato Spring Boot (tramite le opzioni del client factory), garantisce la resilienza della comunicazione in caso di interruzioni temporanee del broker.

4.4 MongoDB

MongoDB è il database NoSQL orientato ai documenti impiegato per la persistenza di tutti i dati della piattaforma.

Nel contesto di ChillChain, la flessibilità di MongoDB si è rivelata particolarmente vantaggiosa per la gestione dei dati telemetrici, la cui struttura può variare in funzione del tipo di sensore, e per le entità di dominio come viaggi e spedizioni, che includono campi con struttura annidata (coordinate geografiche, polyline). L'interazione avviene tramite Spring Data Mon-

goDB, che mappa automaticamente le classi Java alle collezioni corrispondenti. Un aspetto specifico del progetto è l'uso delle pipeline di aggregazione di MongoDB per implementare la logica di ricerca dei viaggi già programmati.

4.5 Python, FastAPI e PyTorch

Python è il linguaggio utilizzato per i due microservizi dedicati rispettivamente allo streaming dei dati telemetrici e alla rilevazione delle anomalie. La scelta è motivata dalla ricchezza dell'ecosistema Python per il *data processing* e il *machine learning*, che ha consentito di implementare entrambi i servizi con un numero ridotto di dipendenze.

Il servizio di streaming (`streamer_main.py`) è un'applicazione FastAPI che legge i file CSV contenenti i dati simulati di telemetria e li pubblica sul broker MQTT con una cadenza configurabile, simulando il comportamento di sensori reali installati a bordo dei veicoli. I dataset sono generati da uno script di simulazione termodinamica (`stream_fridge_dataset_generator.py`), derivato dal dataset Kaggle *Simulated Refrigerator Fault Diagnosis* [5], che modella il comportamento fisico di un impianto frigorifero.

Il servizio di anomaly detection (`anomaly_main.py`) è anch'esso un'applicazione FastAPI che sottoscrive il topic telemetria via MQTT, analizza i dati in ingresso e pubblica i risultati sul topic anomalie. Il cuore del servizio è un modello LSTM Autoencoder implementato in PyTorch: l'encoder, una rete LSTM, comprime la sequenza temporale di ingresso in una rappresentazione latente a dimensionalità ridotta; il decoder, un'altra rete LSTM, ricostruisce la sequenza originale a partire da tale rappresentazione. L'errore di ricostruzione, calcolato come errore quadratico medio (MSE) tra input e output, viene confrontato con una soglia appresa durante l'addestramento su dati di funzionamento normale. Lo scaler, serializzato con Pickle, normalizza le quindici feature in ingresso per garantire che parametri con scale numeriche molto diverse contribuiscano in modo equilibrato all'errore di ricostruzione. La libreria Pandas è impiegata per la lettura dei CSV e la preparazione dei DataFrame, mentre NumPy gestisce le operazioni numeriche sui buffer a finestra scorrevole.

4.6 Google Maps Platform

Le API di Google Maps Platform sono utilizzate per il calcolo dei percorsi, la geolocalizzazione degli indirizzi e l'assistenza all'inserimento degli stessi

da parte degli utenti. Il sistema impiega tre servizi distinti, distribuiti tra frontend e backend.

Sul lato frontend, il componente di inserimento spedizione integra la *Places Autocomplete API*, che fornisce suggerimenti in tempo reale durante la digitazione degli indirizzi di ritiro e consegna. Oltre a migliorare l'esperienza utente riducendo errori di battitura e ambiguità, il servizio restituisce direttamente le coordinate geografiche associate all'indirizzo selezionato, eliminando la necessità di una successiva chiamata di geocodifica.

Sul lato backend vengono utilizzate due API aggiuntive: la *Geocoding API*, che converte gli indirizzi testuali in coordinate geografiche (latitudine e longitudine) nei casi in cui le coordinate non siano già disponibili dal frontend, e la *Routes API*, che calcola il percorso ottimale tra due o più punti restituendo la distanza in metri, la durata stimata e la polyline codificata del tracciato.

Nel flusso operativo di ChillChain, queste API vengono invocate in due momenti: alla creazione di una nuova tratta da parte dell'amministratore, per calcolare il percorso base tra partenza e arrivo, e all'inserimento di una nuova spedizione in un viaggio già esistente, per ricalcolare il percorso con i waypoint intermedi corrispondenti ai punti di ritiro e consegna del nuovo pacco.

4.7 Docker e Docker Compose

Docker è la piattaforma di containerizzazione impiegata per isolare ciascun componente dell'architettura in un ambiente di esecuzione autonomo e riproducibile. Ogni servizio della piattaforma viene eseguito all'interno di un container dedicato, con le proprie dipendenze e la propria configurazione.

Docker Compose orchestra l'avvio coordinato di tutti i container, definendo in un unico file YAML le immagini, le porte esposte, le variabili d'ambiente, i volumi persistenti e le reti interne.

Capitolo 5

Soluzione Proposta

In questo capitolo viene descritta in dettaglio la soluzione realizzata, partendo dall'architettura complessiva del sistema per poi approfondire il funzionamento logico di ciascun componente e i principali dettagli implementativi. La piattaforma ChillChain si compone di tre macro-funzionalità integrate: un sistema di gestione della flotta e dei viaggi riservato all'amministratore, un marketplace per il trasporto condiviso refrigerato e non, e un sistema di monitoraggio in tempo reale con rilevazione automatica delle anomalie.

5.1 Architettura del sistema

L'architettura di ChillChain segue il paradigma a microservizi, in cui ciascun componente è responsabile di una funzione specifica e comunica con gli altri attraverso interfacce ben definite. La Figura 5.1 illustra schematicamente i servizi e i flussi di comunicazione.

Il sistema è composto da sei servizi containerizzati, orchestrati tramite Docker Compose:

Il **backend Spring Boot** (porta 8081) rappresenta il nucleo applicativo: espone le API REST per la gestione di veicoli, viaggi, spedizioni, utenti e notifiche, implementa l'autenticazione JWT e la logica di autorizzazione basata sui ruoli, sottoscrive i topic MQTT per ricevere telemetria e risultati dell'anomaly detection, e persiste tutti i dati su MongoDB.

Il **frontend React** (porta 5173) fornisce l'interfaccia utente, differenziata per ruolo: l'amministratore gestisce la flotta e i viaggi, il cliente cerca e prenota spedizioni, il tecnico monitora i veicoli e riceve le notifiche di anomalia.

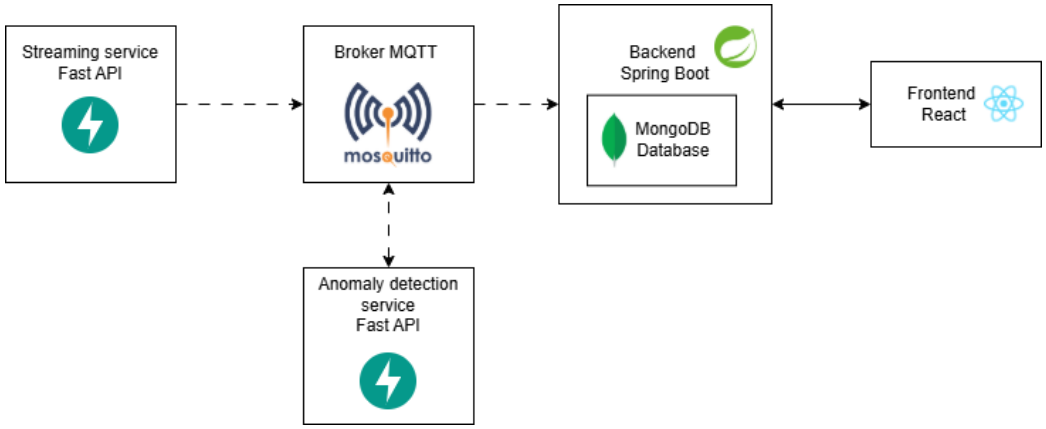


Figura 5.1: Architettura a microservizi di ChillChain. Le frecce continue rappresentano comunicazioni HTTP/REST, le frecce tratteggiate i flussi MQTT.

Il **broker MQTT Mosquitto** (porta 1883) gestisce la comunicazione asincrona tra i servizi Python e il backend, instradando i messaggi attraverso topic.

Il **servizio di streaming** (porta 8002) simula i sensori a bordo dei veicoli, leggendo dataset CSV e pubblicando i dati telemetrici sul broker MQTT con una cadenza di un messaggio al secondo.

Il **servizio di anomaly detection** (porta 8003) sottoscrive la telemetria pubblicata dallo streamer, la analizza con il modello LSTM Autoencoder e pubblica i risultati su un topic dedicato.

Infine, **MongoDB** (porta 27017) funge da database centralizzato, organizzato in sei collezioni: `user`, `vehicle`, `trip`, `shipment`, `telemetry` e `notification`.

5.2 Gestione degli utenti e autenticazione

Il sistema prevede tre ruoli distinti, ciascuno con un perimetro di azioni e una vista dell'interfaccia dedicati:

- **ADMIN**: gestisce la flotta (aggiunta e rimozione di veicoli), visualizza l'elenco dei viaggi programmati.
- **CLIENT**: ricerca viaggi compatibili con le proprie esigenze di spedizione, confronta le opzioni disponibili (inizializzazione di un nuovo viaggio o condividere un viaggio già esistente con altre spedizioni) e prenota la spedizione.

- **TECHNICIAN**: accede alla dashboard di monitoraggio, visualizza lo storico telemetrico dei veicoli e gestisce le notifiche di anomalia.

L'autenticazione è implementata tramite token JWT (*JSON Web Token*). All'atto del login, il backend verifica le credenziali — la password è memorizzata con hashing Argon2 — e genera un token firmato contenente l'email dell'utente e il suo ruolo. Il token viene incluso nell'header `Authorization` di ogni richiesta successiva e validato da un filtro che precede l'elaborazione della richiesta stessa. La catena di sicurezza Spring Security definisce quali endpoint sono accessibili a ciascun ruolo: ad esempio, le operazioni di gestione della flotta richiedono il ruolo `ADMIN`, mentre l'accesso alla telemetria e alle notifiche è riservato al ruolo `TECHNICIAN`. Gli endpoint di autenticazione e di prenotazione delle spedizioni sono invece pubblici.

5.3 Gestione della flotta e dei viaggi

L'amministratore dispone di tre schermate dedicate alla gestione operativa della piattaforma: la registrazione di nuovi veicoli, la consultazione della flotta e il monitoraggio dei viaggi programmati.

5.3.1 Aggiunta di un veicolo

Il form di registrazione raccoglie i dati caratteristici del veicolo: nome identificativo, dimensioni del vano di carico (lunghezza, larghezza e altezza in centimetri), portata massima in chilogrammi, presenza di un impianto di refrigerazione e costo chilometrico in euro. Il frontend esegue una validazione preliminare che impedisce l'invio del form se uno qualsiasi dei campi risulta vuoto o non valido. Al completamento, il backend verifica l'unicità del nome e persiste il documento nella collezione `vehicle` di MongoDB. In caso di successo, l'interfaccia mostra un modale di conferma e azzera tutti i campi, consentendo l'immediata registrazione di un ulteriore mezzo.

5.3.2 Lista veicoli

La schermata di gestione della flotta elenca tutti i veicoli registrati in forma tabellare, esponendo per ciascuno le dimensioni, la portata, il prezzo per chilometro e la presenza della refrigerazione. Per ogni veicolo è disponibile un'azione di eliminazione: prima di procedere, l'interfaccia richiede una conferma esplicita tramite un modale dedicato. Lato backend, l'operazione

è implementata come cancellazione a cascata: vengono rimossi, nell'ordine, tutti gli shipment associati al veicolo, il trip corrispondente e infine il documento veicolo stesso, garantendo la coerenza referenziale dei dati.

5.3.3 Lista viaggi

La schermata dei viaggi presenta tutti i viaggi in programma con le informazioni principali: veicolo assegnato, data di arrivo prevista, distanza totale del percorso e capacità residua. Ciascuna riga è espandibile: selezionando un viaggio, il sistema recupera dal backend la lista degli shipment associati e li mostra in una sotto-tabella con indirizzo di partenza, indirizzo di arrivo, dimensioni del pacco e prezzo. Da questa vista l'amministratore può eliminare l'intero trip con tutti gli shipment collegati.

Per ogni viaggio è inoltre disponibile un'anteprima cartografica: un popup mostra il percorso complessivo decodificato dalla polyline del viaggio su Google Maps. Analogamente, ciascuno shipment espone una propria anteprima cartografica che evidenzia il tratto specifico della spedizione, con marcatori distinti per il punto di ritiro e quello di consegna.

5.4 Marketplace: ricerca dei viaggi disponibili e creazione della spedizione

Il cuore commerciale della piattaforma è il meccanismo che consente ai clienti di creare spedizioni.

5.4.1 Fase 1 — Ricerca dei veicoli disponibili

Quando un cliente inserisce i dati della propria spedizione (indirizzo di partenza e arrivo, dimensioni, peso, necessità di refrigerazione e data di arrivo desiderata), il backend esegue una prima query per individuare i veicoli della flotta che soddisfano i requisiti e che non hanno ancora alcuna viaggio assegnato. Questa query è implementata come aggregation pipeline su MongoDB: il primo stadio filtra i veicoli per peso massimo, volume disponibile e tipologia di refrigerazione; il secondo esegue un `$lookup` sulla collezione `trip`; il terzo seleziona solo i veicoli il cui array di viaggi (trip) risulta vuoto. Per ciascun veicolo disponibile, il sistema invoca la Routes API di Google Maps per calcolare il percorso tra la partenza e l'arrivo indicati dal cliente, ottenendo distanza, durata e polyline codificata. Il prezzo viene calcolato come prezzo = distanceKm $\times$ pricePerKm, dove `pricePerKm` è il costo chilometrico associato al veicolo.

5.4.2 Fase 2 — Ricerca dei viaggi già programmati

Parallelamente, il sistema cerca viaggi già programmati che abbiano capacità residua sufficiente e data di arrivo compatibile. Anche questa ricerca è implementata come aggregation pipeline: il primo stadio esegue un `$lookup` con la collezione `vehicle` per recuperare la tipologia di refrigerazione; il secondo filtra per capacità residua, refrigerazione e data.

5.4.3 Fase 3 — Validazione geografica dei viaggi già programmati

I viaggi già programmati individuati nella fase precedente vengono ulteriormente filtrati verificando la compatibilità geografica con la spedizione richiesta. Per ciascun viaggio candidato, il sistema decodifica la polyline del percorso e calcola la distanza geodetica (formula di Haversine) tra i punti di ritiro e consegna del nuovo pacco e il tracciato del viaggio. Se entrambi i punti distano meno di 1,5 km dalla polyline, la tratta è considerata geograficamente compatibile. Un ulteriore controllo verifica che il punto di ritiro preceda quello di consegna lungo il percorso, impedendo deviazioni illogiche.

5.4.4 Fase 4 — Ricalcolo del percorso con waypoint

Per le tratte che superano la validazione geografica, il sistema ricalcola il percorso includendo i waypoint intermedi di tutti gli shipment (sia quelli già assegnati sia quello nuovo). L'ordinamento dei waypoint è affidato a un algoritmo greedy con vincoli: partendo dalla posizione corrente, seleziona il waypoint disponibile più vicino, rispettando il vincolo che il punto di ritiro di ciascuna spedizione debba essere visitato prima del corrispondente punto di consegna. Un delivery diventa "disponibile" solo dopo che il relativo pickup è stato inserito nella sequenza. Il percorso ricalcolato viene confrontato con quello originale: se la differenza di durata non supera i 30 minuti, la tratta viene inclusa tra i risultati.

5.4.5 Fase 5 — Pricing con divisore esponenziale

Il sistema di pricing incentiva la condivisione attraverso un divisore esponenziale. Se il veicolo ospita già n spedizioni, il prezzo per il nuovo cliente viene calcolato come:

$$\text{prezzo} = \frac{\text{distanceKm}_{\text{shipment}} \times \text{pricePerKm}_{\text{vehicle}}}{2^n} \tag{1}$$

Dove $\text{distanceKm}_{\text{shipment}}$ è la distanza del tratto specifico del nuovo cliente e n è il numero di spedizioni già presenti. In questo modo, la prima spedizione paga il prezzo pieno, la seconda paga metà, la terza un quarto e così via. Questo meccanismo crea un incentivo economico crescente: più il veicolo è occupato, più il trasporto diventa conveniente per i nuovi clienti, incentivando il riempimento del mezzo e riducendo i costi per tutti i partecipanti.

5.4.6 Selezione e conferma della tratta

Il cliente visualizza tutte le opzioni — viaggi nuovi e viaggi già esistenti. Al momento della selezione, il backend esegue l'operazione seguente: se il viaggio è nuovo (veicolo senza trip assegnato), viene creata una nuova entità `Trip`; se il viaggio è già esistente, viene aggiornata l'entità `Trip` esistente con il nuovo percorso (inclusivo dei waypoint) e la capacità residua aggiornata. In entrambi i casi, viene creata un'entità `Shipment` associato al viaggio.

5.5 Generazione dei dati telemetrici

La simulazione della telemetria di bordo si basa su un modello termodinamico che riproduce il comportamento fisico di un impianto frigorifero a compressione di vapore. Il generatore, derivato dal dataset Kaggle *Simulated Refrigerator Fault Diagnosis* [5], produce dataset CSV contenenti 120 campioni da 15 feature ciascuno (un campione rappresenta dati estratti in un certo minuto), strutturati in due fasi: i primi 60 minuti in condizioni normali, seguiti da 60 minuti con un guasto iniettato. Sono previste varie tipologie di guasto, tra cui: fouling del condensatore (lieve e severo), degradazione e guasto della ventola dell'evaporatore, ecc.

Il servizio di streaming legge questi CSV e pubblica ciascuna riga sul topic MQTT `fridge/{vehicleName}/telemetry` con un intervallo configurabile (default: 1 secondo), includendo un timestamp ISO e un indice di riga progressivo. Al termine del dataset, pubblica un messaggio di completamento con `stream_status: "completed"`, che segnala agli altri servizi la fine dello stream.

5.6 Pipeline di anomaly detection

La rilevazione delle anomalie si articola in tre fasi: l'addestramento offline del modello, l'inferenza in tempo reale e il meccanismo di smoothing per la riduzione dei falsi positivi.

5.6.1 Addestramento del modello

Il modello LSTM Autoencoder è stato addestrato su Google Colab con GPU Tesla T4, utilizzando un dataset composto esclusivamente da dati di funzionamento normale. L'architettura del modello è la seguente: un encoder LSTM con 15 feature in ingresso e hidden size pari a 16, uno strato lineare di compressione che riduce la rappresentazione a uno spazio latente di dimensione 8, due strati lineari che inizializzano hidden state e cell state del decoder a partire dalla rappresentazione latente, un decoder LSTM con hidden size 16 alimentato da una sequenza di zeri (la ricostruzione dipende interamente dallo stato latente) e uno strato lineare di output che proietta la dimensione nascosta nelle 15 feature originali.

La soglia di anomalia è stata determinata calcolando l'MSE di ricostruzione su tutto il dataset di training normale e prendendo il 95° percentile della distribuzione risultante (threshold $= 0,009868$).

5.6.2 Inferenza in tempo reale

Il servizio di anomaly detection sottoscrive il topic `fridge/+/telemetry` tramite il broker MQTT, ricevendo i dati di tutti i veicoli. Per ciascun veicolo viene creato dinamicamente un'istanza di `VehicleAnomalyDetector` alla ricezione del primo messaggio, garantendo il supporto simultaneo di più veicoli con buffer e contatori indipendenti.

Ad ogni messaggio ricevuto, le 15 feature vengono estratte, normalizzate con lo scaler e inserite in un buffer a finestra scorrevole di 30 campioni. Quando il buffer è pieno, la finestra viene convertita in un tensore PyTorch e passata al modello in modalità di inferenza (`torch.no_grad()`). L'errore di ricostruzione, calcolato come MSE tra input e output mediato su tutti i timestep e tutte le feature, viene confrontato con la soglia. Al completamento dello stream, il detector del veicolo viene rimosso e le sue risorse liberate.

5.6.3 Smoothing con contatori consecutivi

Per evitare che oscillazioni transitorie dell'errore di ricostruzione generino falsi allarmi, il sistema implementa un meccanismo di smoothing basato su contatori consecutivi. Vengono mantenuti due contatori per ciascun veicolo: un `anomaly_counter`, che si incrementa ad ogni campione con MSE $>$ threshold e si azzera al primo campione normale, e un `normal_counter`, che opera in modo simmetrico. La transizione dallo stato normale allo stato di anomalia avviene solo quando il contatore di anomalie supera la soglia di 20 campioni consecutivi; analogamente, il ritorno allo stato normale richiede 20 campioni

consecutivi sotto soglia. Questo meccanismo introduce un ritardo controllato nella segnalazione (20 minuti con campionamento al minuto), ma elimina le notifiche spurie causate da picchi isolati, come quelli associati alle aperture della porta o ai cicli di sbrinamento.

Il risultato dell'analisi viene pubblicato sul topic `fridge/{vehicleName}/anomalies`, includendo l'errore di ricostruzione, lo stato dei contatori e, nelle sole transizioni di stato, un messaggio di allerta che il backend persisterà come notifica.

5.7 Gestione delle notifiche

Il backend sottoscrive il topic `fridge/+/anomalies` tramite un channel adapter Spring Integration dedicato. Alla ricezione di un messaggio di anomalia, il backend verifica la presenza del campo `alertMessage`: se presente, indica una transizione di stato (da normale ad anomalia) e genera una nuova notifica persistente su MongoDB con i campi: nome del veicolo, timestamp, errore di ricostruzione, indice di riga e messaggio di allerta. L'interfaccia del tecnico presenta le notifiche in ordine cronologico con un badge numerico che indica il conteggio delle notifiche non lette. Ciascuna notifica può essere contrassegnata come letta tramite un'azione dedicata, aggiornando lo stato sul database.

5.8 Dashboard telemetrica

La dashboard di monitoraggio consente al tecnico di visualizzare lo storico telemetrico dei veicoli tramite grafici temporali interattivi. L'interfaccia è organizzata in due livelli: un selettore del veicolo, che elenca i veicoli con refrigerazione, e un pannello di visualizzazione con grafici ad area realizzati tramite Recharts.

Per ciascun veicolo selezionato, il frontend richiede al backend l'elenco completo dei dati telemetrici memorizzati, ordinati cronologicamente. Il tecnico può filtrare la visualizzazione per categoria di parametro (ad esempio, temperature o pressioni) e per intervallo temporale.

Capitolo 6

Validazione Funzionale

In questo capitolo viene descritta la validazione funzionale della piattaforma ChillChain attraverso uno scenario d'uso completo che attraversa tutte le funzionalità del sistema, coinvolgendo i tre ruoli previsti: amministratore, cliente e tecnico. Lo scenario simula un caso realistico in cui un'azienda di trasporti refrigerati gestisce una flotta di veicoli, riceve richieste di spedizione da più clienti e rileva un'anomalia sull'impianto frigorifero durante il viaggio.

6.1 Ambiente di test

La validazione è stata condotta su una macchina locale con l'intero sistema eseguito in container Docker. Il frontend è raggiungibile all'indirizzo `http://localhost:5173`.

Per la simulazione della telemetria sono stati predisposti dataset CSV generati dallo script di simulazione termodinamica descritto nella Sezione 5.5: un dataset in condizioni di funzionamento normale per 120 minuti e un dataset con iniezione del guasto `UNDERCHARGE_AND_COND_FOUL` (sottocaricamento del refrigerante combinato a fouling del condensatore) a partire dal minuto 60. Entrambi i dataset sono composti da 120 righe con campionamento al minuto.

6.2 Scenario di validazione

Lo scenario è strutturato in cinque fasi che coprono l'intero ciclo di vita di una spedizione, dalla configurazione della flotta fino alla rilevazione di un'anomalia in corso di viaggio.

6.2.1 Fase 1 — Configurazione della flotta (Amministratore)

L'amministratore accede alla piattaforma con le proprie credenziali tramite il login form mostrato in Figura 6.1. Procede, poi, dalla sezione *Add Vehicle* alla registrazione di tre veicoli dalla schermata di registrazione veicoli (Figura 6.2).

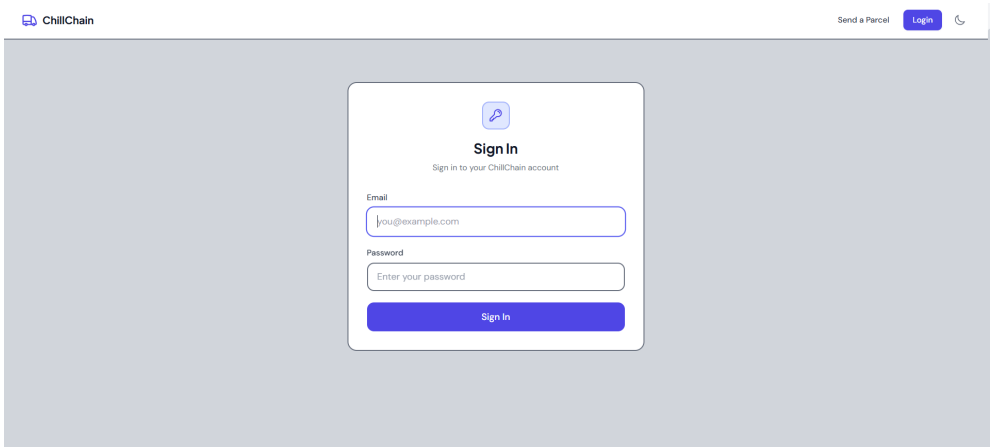


Figura 6.1: Schermata di login della piattaforma. L'utente inserisce email e password; il sistema restituisce un token JWT e fa accedere all'interfaccia corrispondente al ruolo.

Il primo veicolo, denominato `camion1`, è configurato con un vano di carico di $4 \times 2 \times 2$ m, peso massimo 1000 kg, refrigerazione attiva e costo di 1 €/km. Il secondo veicolo, `camion2`, ha dimensioni $3 \times 1.5 \times 1.5$ m, peso massimo 700 kg e un costo di 0,40 €/km, non dispone di refrigerazione. Il terzo veicolo, `camion3`, ha dimensioni $5 \times 2 \times 2$ m, peso massimo 1200 kg e un costo di 1 €/km, dispone di refrigerazione. Nella sezione *Vehicles* (Figura 6.3) è possibile visualizzare la lista dei veicoli registrati, con le relative caratteristiche.

6.2.2 Fase 2 — Prima spedizione: creazione della tratta (Cliente A)

Un primo cliente va alla sezione *Send Parcel* (Figura 6.4) e inserisce i dati della propria spedizione: partenza da Milano, arrivo a Roma, pacco di $50 \times 50 \times 50$ cm con peso di 100 kg, refrigerazione necessaria e data di arrivo entro il 20 marzo 2026.

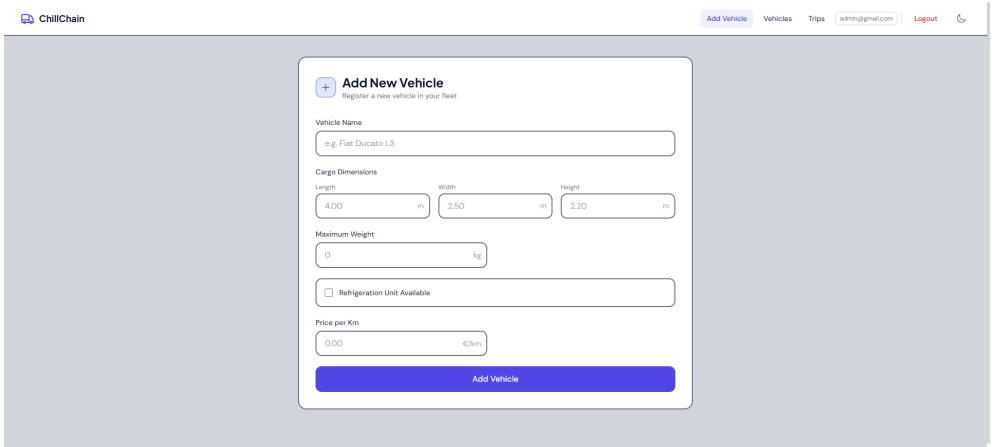


Figura 6.2: Form di registrazione di un nuovo veicolo. L'amministratore specifica il nome identificativo, le dimensioni del vano di carico (lunghezza, larghezza, altezza in m), il peso massimo trasportabile (kg), se è disponibile la refrigerazione e il costo per chilometro (€/km).

Il sistema restituisce la lista delle tratte disponibili (Figura 6.5). In questo caso, poiché nessun veicolo ha ancora una tratta assegnata, vengono mostrati entrambi i veicoli come opzioni "vuote": per entrambi 587,89 € (calcolati come distanza Milano–Roma × prezzo/km del rispettivo veicolo).

Il Cliente A seleziona `camion3`. Il backend crea una nuova entità `Trip` con il percorso Milano–Roma, decrementa la capacità residua del veicolo e salva lo `Shipment` associato.

6.2.3 Fase 3 — Seconda spedizione: condivisione della tratta (Cliente B)

Un secondo cliente effettua una ricerca per una spedizione da Modena a Firenze, con un pacco più piccolo ($30 \times 30 \times 30$ cm, 20 kg, refrigerato, arrivo massimo il 22 marzo 2026).

Il sistema mostra due opzioni (Figura 6.6): `camion1` per un viaggio nuovo (Modena–Firenze a 142,03 €), `camion1` per un viaggio condiviso (il percorso ricalcolato Milano–Modena–Firenze–Roma rientra nella tolleranza di 1 ora, il è prezzo scontato grazie al divisore 2^1). Il Cliente B seleziona il viaggio in condivisione. Il backend aggiorna la polyline del trip con i nuovi waypoint (Modena e Firenze come tappe intermedie), decrementa ulteriormente la capacità residua e salva il nuovo `Shipment`.

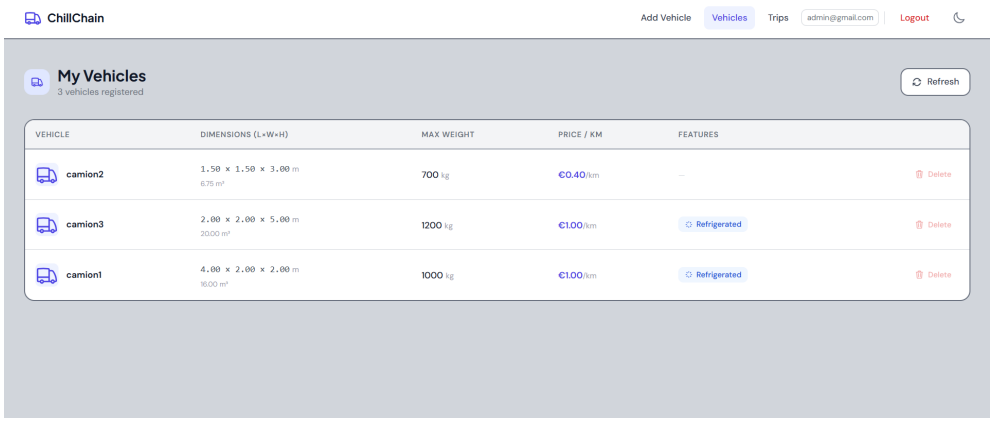


Figura 6.3: Lista dei veicoli registrati nella flotta. Per ciascun veicolo sono visualizzati il nome, le dimensioni, il peso massimo, se è refrigerato e il prezzo per chilometro.

6.2.4 Fase 4 — Visualizzazione dei viaggi programmati e avvio simulazione

L'amministratore può vedere l'elenco dei viaggi programmati dalla sezione *Trips* (Figura 6.7). Per ogni viaggio è possibile visualizzare il percorso in mappa e gli shipment relativi.

Da postman è possibile usare un endpoint del backend per avviare la simulazione della telemetria per `camion3`. Il backend inoltra la richiesta al servizio di streaming, che inizia a pubblicare i dati sul topic `fridge/camion3/telemetry`.

6.2.5 Fase 5 — Rilevazione anomalia e notifica (Tecnico)

Il tecnico accede alla piattaforma con le proprie credenziali e viene indirizzato all'elenco dei veicoli dotati di refrigerazione (Figura 6.8).

Durante i primi 60 minuti simulati, il modello LSTM Autoencoder ricostruisce correttamente le sequenze. A partire dal minuto 60 il dataset è influenzato dal guasto `UNDERCHARGE_AND_COND_FOUL`. L'autoencoder inizia a produrre errori di ricostruzione superiori alla soglia. Dopo 20 campioni consecutivi anomali, il servizio di anomaly detection pubblica un messaggio di allerta sul topic `fridge/camion3/anomalies`.

Il backend riceve il messaggio e crea una notifica persistente . L'icona delle notifiche nell'header si aggiorna con il badge numerico. Il tecnico seleziona la notifica e accede al pannello notifiche (Figura 6.9).

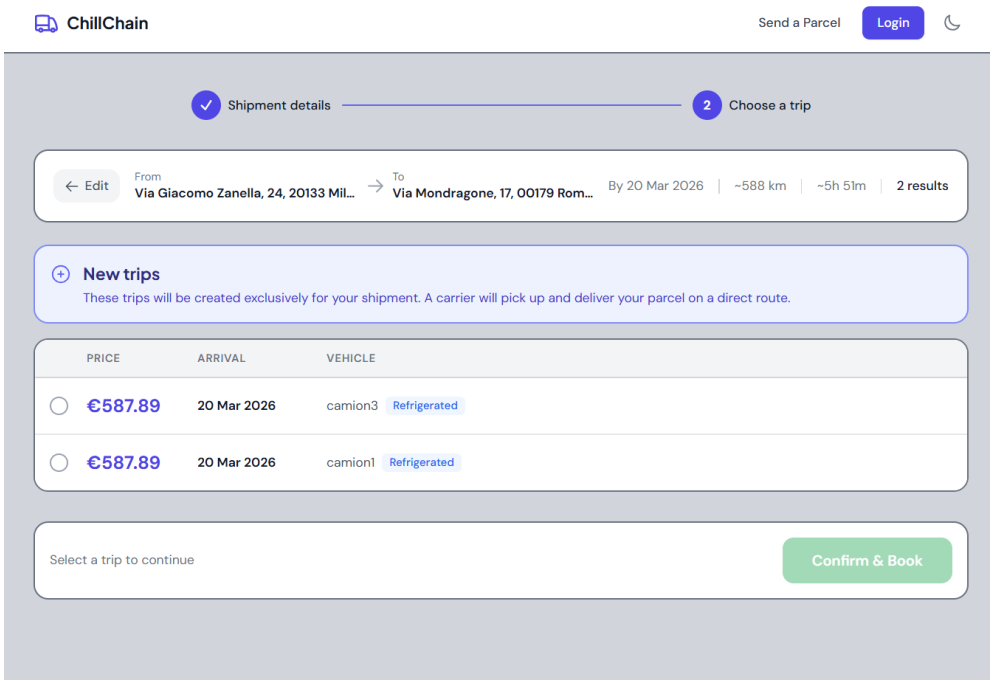


Figura 6.4: Form di ricerca tratte. Il cliente inserisce gli indirizzi di partenza e arrivo, le dimensioni e il peso del pacco, la necessità di refrigerazione e la data di arrivo desiderata.

Dall'elenco dei veicoli il tecnico seleziona `camion3` e accede alla dashboard telemetrica (Figura 6.10), dove può visualizzare i grafici storici dei parametri monitorati.

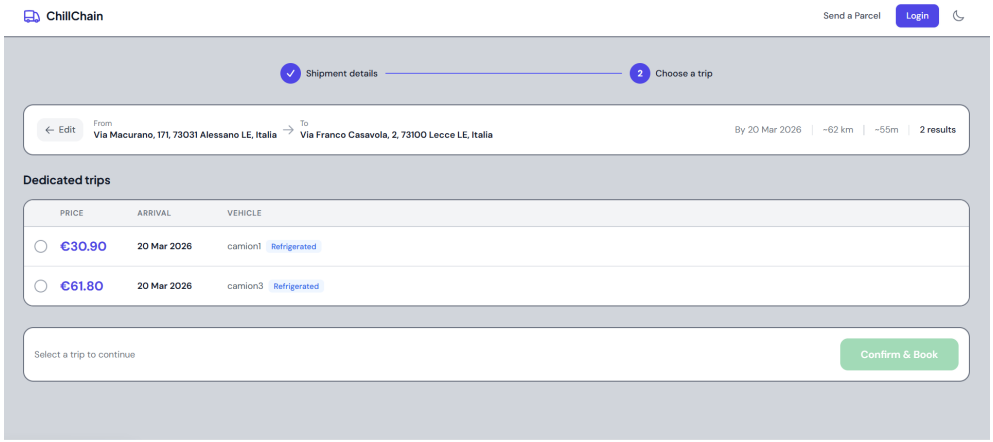


Figura 6.5: Risultati della ricerca tratte per il Cliente A. Sono visibili due veicoli disponibili con il percorso Milano–Roma, prezzo calcolato, distanza e durata stimata.

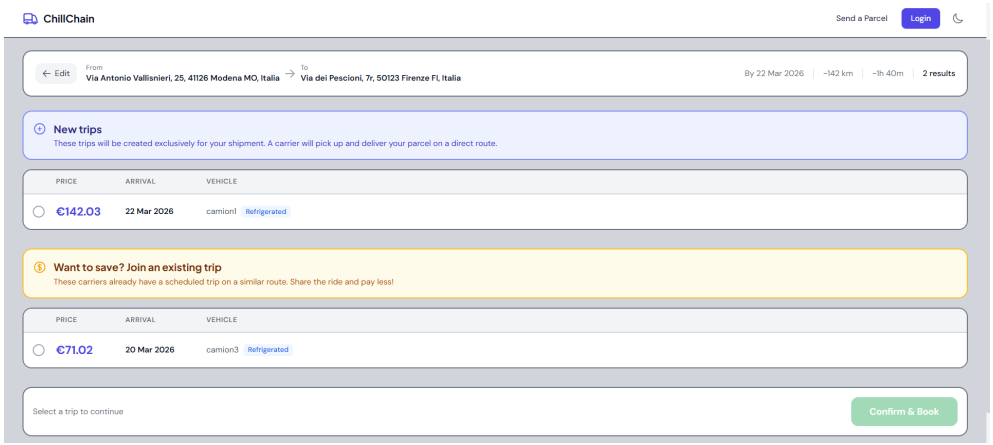
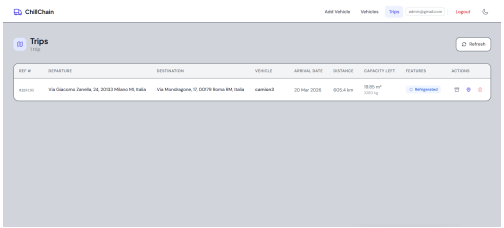
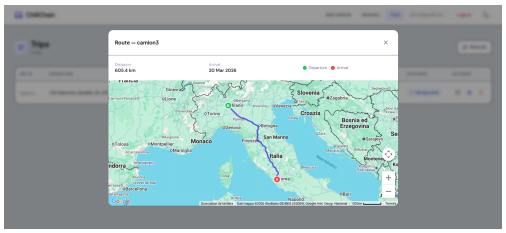


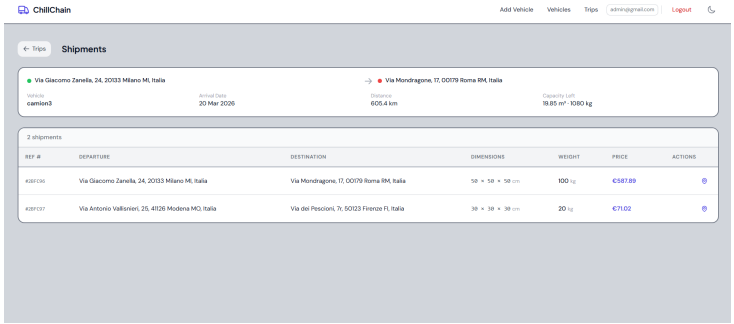
Figura 6.6: Risultati della ricerca tratte per il Cliente B. Oltre al veicolo `camion1` disponibile per un viaggio nuovo, compare `camion3` per un viaggio condiviso: il percorso Milano–Roma passa sufficientemente vicino a Modena e Firenze, la deviazione rientra nell'ora di tolleranza e il prezzo è scontato del 50%



(a) Lista dei viaggi programmati.



(b) Mappa del percorso del viaggio selezionato.



(c) Dettaglio del viaggio con elenco degli shipment associati.

Figura 6.7: Descrizione generale della figura

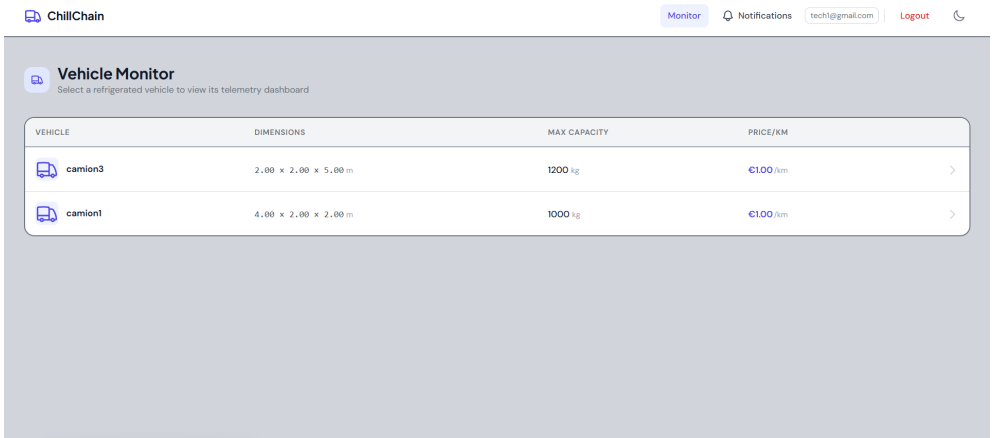


Figura 6.8: Elenco dei veicoli dotati di refrigerazione.

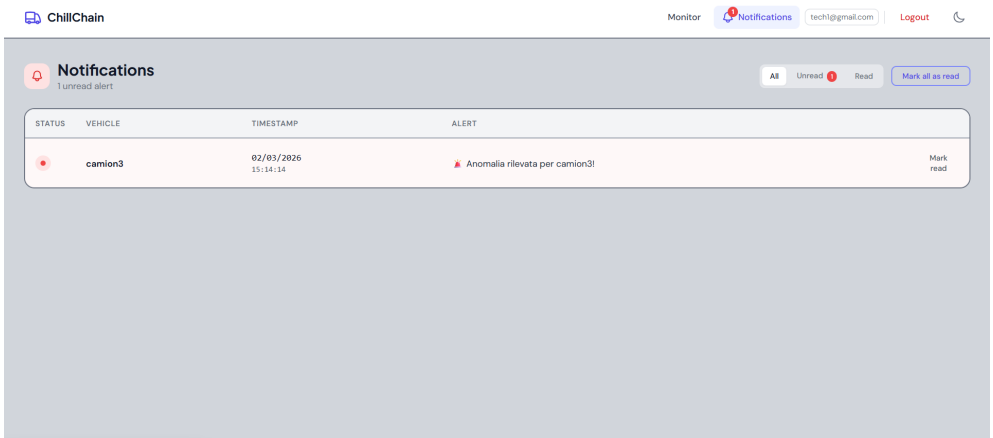


Figura 6.9: Pannello notifiche del tecnico. La notifica indica il veicolo interessato (`camion3`), il timestamp dell'allerta e il messaggio. Il badge nell'header mostra il conteggio delle notifiche non lette.

Figura 6.10: Dashboard telemetrica del veicolo `camion3`

Capitolo 7

Conclusioni e Sviluppi Futuri

7.1 Conclusioni

L'obiettivo di questo lavoro era la progettazione e l'implementazione di una piattaforma IoT integrata per il trasporto refrigerato condiviso, capace di unire in un unico sistema due funzionalità tradizionalmente separate: un marketplace per il matching tra domanda e offerta di spedizioni e un sistema di monitoraggio delle condizioni del vano di refrigerazione dei veicoli con rilevazione automatica delle anomalie.

7.2 Sviluppi futuri

Nonostante i risultati ottenuti, il lavoro presenta margini di miglioramento che possono essere affrontati in sviluppi successivi.

7.2.1 Ottimizzazione temporale e scheduling attivo

L'attuale algoritmo di matching tra spedizioni e viaggi opera prevalentemente sulla dimensione spaziale, valutando la prossimità geografica tra i punti di ritiro/consegna e la polyline del percorso. La dimensione temporale, pur presente sotto forma di data di scadenza della spedizione, non viene sfruttata in modo strutturato per guidare la pianificazione dei viaggi. Un sistema più maturo potrebbe integrare le finestre di disponibilità dei trasportatori, i periodi di inattività dei veicoli e i vincoli temporali delle spedizioni in un unico modello di ottimizzazione.

7.2.2 Scalabilità su piattaforme cloud

L'attuale architettura è validata in ambiente locale tramite Docker Compose. Per un utilizzo in produzione sarebbe necessario il deployment su piattaforme cloud come AWS o Azure, sfruttando servizi gestiti quali container orchestration (ECS/EKS), database managed (MongoDB Atlas), e broker MQTT gestiti (AWS IoT Core). Questo garantirebbe alta disponibilità, auto-scaling in funzione del carico e ridondanza geografica.

7.2.3 Miglioramento della sensibilità dell'anomaly detection

Il modello LSTM autoencoder attuale opera sulle 15 feature grezze del dataset di telemetria refrigerata. Anomalie sottili come il *sensor drift*, in cui un sensore si degrada progressivamente nel tempo senza generare un errore di ricostruzione elevato su singole osservazioni, risultano difficili da rilevare con questo approccio.

Un miglioramento consiste nell'estrazione di feature derivate da fornire in input al modello come il rapporto tra feature correlate.

7.2.4 Indicizzazione spaziale con PostGIS

Il calcolo della distanza punto-polyline, utilizzato per determinare la compatibilità tra un punto di ritiro/consegna e un percorso esistente, avviene attualmente con una scansione lineare $O(n)$ su tutti i segmenti della polyline decodificata. Questa complessità diventa un collo di bottiglia all'aumentare del numero di viaggi e della lunghezza dei percorsi.

L'adozione di PostgreSQL con l'estensione PostGIS consentirebbe di sfruttare l'indicizzazione spaziale tramite R-tree: la funzione `ST_Distance` opera su geometrie indicizzate con complessità $O(\log n)$, riducendo drasticamente il tempo di ricerca. Inoltre, PostGIS offre nativamente operazioni come il buffer spaziale e l'intersezione tra geometrie.

7.2.5 Portale cliente per il tracking delle spedizioni

Attualmente il ruolo `CLIENT` è limitato alla fase di ricerca e prenotazione. Una volta confermata la spedizione, il cliente non dispone di strumenti per monitorarne lo stato. Un'evoluzione prevede la creazione di un'area riservata in cui il cliente possa visualizzare lo stato della spedizione in tempo reale, con stima dell'orario di arrivo (ETA) calcolata sulla base della posizione corrente

del veicolo e della rotta pianificata, e notifiche automatiche all'approssimarsi del ritiro o della consegna.

7.2.6 Sistema di notifiche push per il tecnico

Il sistema attuale di notifiche per anomalie è consultabile esclusivamente tramite la dashboard web: il tecnico deve accedere alla piattaforma per verificare la presenza di nuovi alert. In un contesto operativo reale, le anomalie della catena del freddo richiedono interventi tempestivi. L'integrazione di canali di notifica push, tramite email (SMTP o servizi come SendGrid) e notifiche mobile (Firebase Cloud Messaging), garantirebbe che il tecnico venga avvisato immediatamente al verificarsi di un'anomalia, indipendentemente dal fatto che stia consultando la dashboard in quel momento.

Bibliografia

[1] Bilge Atasoy, Frederik Schulte e Alex Steenkamp. «Platform-Based Collaborative Routing using Dynamic Prices as Incentives». In: *Transportation Research Record* 2674.10 (2020), pp. 670–684. DOI: 10.1177/0361198120935116.

[2] Flock Freight. *FlockDirect® Shared Truckload*. 2024. URL: https://www.flockfreight.com (visitato il 24/02/2025).

[3] James Gillespie et al. «Real-Time Anomaly Detection in Cold Chain Transportation Using IoT Technology». In: *Sustainability* 15.3 (2023), p. 2255. DOI: 10.3390/su15032255.

[4] Quicargo B.V. *Digital Freight Platform for Road Transport and Logistics*. 2024. URL: https://quicargo.com (visitato il 24/02/2025).

[5] Mikhail Samoilov. *Simulated Refrigerator Fault Diagnosis Dataset*. 2024. URL: https://www.kaggle.com/datasets/samoilovmikhail/simulated-refrigerator-fault-diagnosis-dataset (visitato il 24/02/2025).

[6] Arnaldo Sgueglia et al. «A systematic literature review of IoT time series anomaly detection solutions». In: *Future Generation Computer Systems* 134 (2022), pp. 170–186. DOI: 10.1016/j.future.2022.04.005.